Benchmarking and Profiling High-Throughput Serving Frameworks for Large Language Models: A Systematic Study of vLLM and SGLang on NVIDIA L4

Yimeng Ye, Can Yang, Yuxuan Wang, Jiawei Wang, Zhijing Wu
COMS 6998 High Performance Machine Learning, Spring 2026
Columbia University
New York, NY, USA
{yy3608, cy2811, yw4609, jw4807, zw3155}@columbia.edu

***Abstract*—In practical deployments of large language models (LLMs), the real bottleneck is usually not raw GPU compute, but how efficiently the serving runtime manages concurrent requests and the key-value (KV) cache. In this project we benchmarked three inference setups on a single NVIDIA L4 (24 GB) GPU: a plain HuggingFace Transformers loop (our sequential baseline), vLLM with PagedAttention, and SGLang with RadixAttention. We tested four models (Qwen2.5-1.5B-Instruct in FP16 and AWQ-INT4, Qwen2.5-3B-Instruct FP16, and Llama-3.2-1B-Instruct FP16) at five concurrency levels ($c \in \{1, 2, 4, 8, 16\}$) on two workloads: short independent Q&A prompts and longer prompts that share a common prefix (designed to exercise prefix caching).**

The headline results: Compared to the HuggingFace baseline, vLLM and SGLang push throughput up by 20–$40\times$ at $c = 16$ on Qwen2.5-1.5B. SGLang's RadixAttention cuts time-to-first-token (TTFT) by $3.5\times$ on shared-prefix prompts (226 ms $\rightarrow$ 64 ms) without hurting throughput. AWQ-INT4 quantization doubled throughput on SGLang (1108 $\rightarrow$ 2280 tok/s), but—and this surprised us—it actually *slowed down* vLLM on the same GPU because of a dequantization kernel overhead issue. We open-source all scripts, raw JSON results, and Weights & Biases dashboards so anyone can reproduce our benchmarks.

***Index Terms*—LLM inference, serving frameworks, vLLM, SGLang, PagedAttention, RadixAttention, KV cache, AWQ quantization, GPU benchmarking.**

I. Introduction

Deploying LLMs in production environments incurs significant inference costs. Two things matter most for efficiency: (1) whether the runtime can overlap many requests at once through continuous batching, and (2) how it handles the ever-growing KV cache that autoregressive decoding produces. The default HuggingFace `generate` call basically processes one request (or one static batch) at a time—if some sequences finish early, the GPU just sits there waiting for the longest one. That is a lot of wasted compute.

Two popular open-source serving frameworks try to fix this. vLLM [1] introduces *PagedAttention*, which manages the KV cache the way an OS manages virtual memory: it breaks the cache into fixed-size pages, tracks them with a page table, and packs them tightly so there is no fragmentation. This makes continuous batching practical even under tight memory budgets. SGLang [2] goes a step further with *RadixAttention*: it organizes cached prefixes into a radix tree, so if multiple requests start with the same text (think system prompts, RAG context, or tool descriptions), the prefill computation is done once and shared across all of them.

To guide the deployment of LLMs on resource-constrained hardware, our evaluation focuses on four key research questions:

1) **How much faster are vLLM and SGLang compared to a naive HuggingFace loop?**
2) **When does RadixAttention actually help, and by how much?**
3) **Does AWQ INT4 quantization translate to real wall-clock speedups, and is the answer the same on both frameworks?**
4) **At the kernel level, where is the time actually going?**

We built a fully automated pipeline that sweeps every combination of (framework, model, quantization, workload, concurrency), logs everything to Weights & Biases [6], and generates the figures in this report. All the numbers here were collected on a fresh L4 instance after our mid-term checkpoint—they replace the earlier L40S numbers from our progress slides. The full codebase is at https://github.com/lennonYe/6998_project.

II. Models and Workloads

A. Models

We picked four small, open-weight chat models. Going small was intentional: everything fits on a single 24 GB L4 without tensor parallelism, so we are measuring the serving framework's efficiency rather than multi-GPU communication overhead. We kept the exact same model weights across all three backends so that any performance difference is purely due to the runtime.

- **Qwen2.5-1.5B-Instruct** (1.54 B params, FP16) [4] — our primary reference model. Most of the detailed plots in this report use this one.

- **Qwen2.5-1.5B-Instruct-AWQ** — same architecture, but with 4-bit AWQ quantization [3]. Weights are compressed to INT4 with per-channel scales; activations stay in FP16.
- **Qwen2.5-3B-Instruct** (3.09 B params, FP16) — a 2× bigger model to see how the results shift with scale.
- **Llama-3.2-1B-Instruct** (1.24 B params, FP16) [5] — a completely different model family, included as a sanity check to make sure our findings are not Qwen-specific.

B. Workloads

Real LLM traffic tends to be bimodal: lots of short, unrelated questions, plus some longer requests that share boilerplate context. We designed two synthetic workloads to capture both patterns.

a) Short Q&A.: 16 single-sentence questions on different topics (CS trivia, geography, math, networking). Each prompt is about 30–60 tokens. Since every prompt is completely different, there is nothing for a prefix cache to reuse. This workload is a pure test of batching efficiency.

b) Shared-prefix code analysis.: A fixed ∼300-token prefix containing a Python class called `DataProcessor`, followed by one of 16 different suffix instructions (e.g., "Focus on bug detection only," "Convert this to use async I/O"). Because the prefix is identical across all requests in a batch, a framework with prefix caching can compute it once and share the result. This is the workload that puts RadixAttention to the test.

For every experiment we set `max_new_tokens=256` and `temperature=0` (greedy decoding), so output lengths are deterministic and any performance differences come from the runtime, not sampling randomness.

C. Hardware and software

All experiments ran on an Ubuntu 22.04 VM with a single NVIDIA L4 (24 GB, FP16 peak ≈ 121 TFLOP/s) and CUDA 12.4. One practical headache: vLLM, SGLang, and HuggingFace each pin different versions of `torch`, `flash-attn`, and `transformers`, so we had to maintain three separate virtual environments (`env_baseline`, `env_vllm`, `env_sglang`). Our orchestration script activates the correct one before each run. Package versions: `vllm 0.6.x`, `sglang 0.4.x`, `transformers 4.46`, `torch 2.4`.

III. Measurement and Profiling Methodology

A. Benchmark driver

We wrote a small Python harness (in `benchmark/scripts/`) that fires prompts and records per-request stats: input/output token counts, TTFT, total wall time, and end-to-end latency. For the HuggingFace baseline we simply call `model.generate` in a loop, since it does not support continuous batching (which is itself one of the things we wanted to demonstrate). For vLLM and SGLang we spin up their HTTP servers and send c concurrent requests using `aiohttp` against the OpenAI-compatible API. Throughput is computed as $\sum_i \text{output_tokens}_i \,/\, \text{wall_time}$ over the whole batch.

B. Metrics

- **TTFT (time-to-first-token):** Wall-clock time from sending the request to receiving the first streamed token. We enable streaming so the server flushes the prefill result immediately, which means TTFT cleanly captures the prefill cost.
- **Throughput:** Total output tokens divided by total wall time for the entire concurrent batch. This is the number that determines your per-token cost and whether you can meet latency SLOs.
- **Latency:** Average end-to-end time per request (prefill + decode).
- **Peak GPU memory:** Read from `nvidia-smi`. As we discuss later (Section V-E), comparing this across frameworks is tricky because vLLM and SGLang pre-allocate nearly all GPU memory for the KV pool at startup.

C. Profiling

To understand *why* the baseline is so slow, we wrapped a single HuggingFace forward pass in the PyTorch Profiler (`torch.profiler.profile` with `ProfilerActivity.CUDA`) and exported the per-kernel timing to a CSV. For vLLM and SGLang we relied on their built-in server logs to verify batch sizes and KV-cache hit rates.

D. Reproducibility

The entire experiment sweep is driven by a single script (`benchmark/run_all_models.sh`) that iterates over all four models and dumps per-model JSON files into `results/<model>/`. A separate `visualize.py` reads those JSONs, generates all the figures, and pushes them to our WandB project (`shvpz8ctzd-1/llm-serving-benchmark`). Each model×quantization combination is a separate WandB run, which made it easy for the team to navigate the results during analysis.

IV. Optimizations Under Study

We looked at three optimization techniques and applied them incrementally so we could see what each one contributes on its own.

a) (O1) Continuous batching with PagedAttention (vLLM).: PagedAttention [1] breaks each request's KV cache into fixed-size pages (16 tokens by default) and tracks them with a page table, similar to how an OS manages virtual memory. Two things matter here: (a) the cache pool is tightly packed with no wasted space, so you can fit more sequences in memory; (b) the scheduler can slot in a new request mid-decode just by allocating fresh pages, which eliminates the head-of-line blocking that kills the HuggingFace baseline. We used vLLM's defaults (`block_size=16`, `gpu_memory_utilization=0.9`).

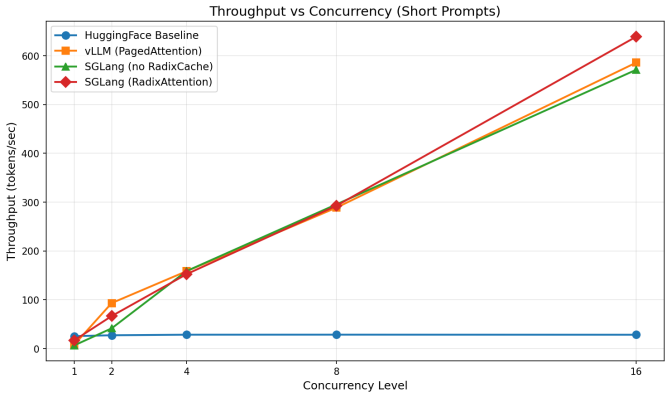

Fig. 1. Throughput vs. concurrency on the short-Q&A workload, Qwen2.5-1.5B-FP16. The HuggingFace baseline flatlines because it cannot do continuous batching, while vLLM and SGLang scale up nicely.

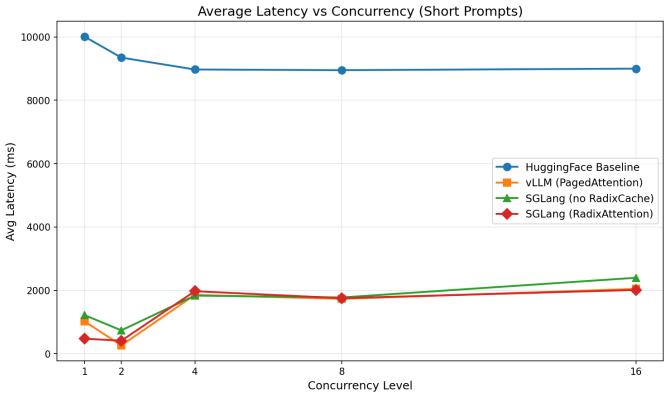

Fig. 2. Per-request latency vs. concurrency on short Q&A, Qwen2.5-1.5B-FP16. Even at 16 concurrent requests, vLLM and SGLang keep per-request latency roughly $4\times$ lower than the sequential baseline.

b) (O2) Prefix sharing with RadixAttention (SGLang).: RadixAttention [2] adds a radix-tree index on top of the KV cache. When a new request comes in, the scheduler walks the tree to find the longest matching prefix and reuses that cached KV state. For our shared-prefix workload, the ~300-token Python class gets computed once by the first request, and every subsequent request gets it for free. We tested SGLang with the radix cache on (`--enable-radix-cache`) and off, running them as two separate server instances back-to-back on the same inputs.

c) (O3) AWQ INT4 weight-only quantization.: AWQ [3] compresses model weights to 4 bits using per-channel scales that are chosen to preserve the most important activation channels. The actual compute still happens in FP16 after on-the-fly dequantization, so the speedup comes entirely from reduced memory bandwidth usage. We tested the AWQ variant of Qwen2.5-1.5B on both vLLM and SGLang. Since the L4 has strong FP16 compute but relatively modest memory bandwidth, it was not obvious ahead of time whether this trade-off would pay off.

A. What we deliberately did not tune

To keep things fair, we (i) fixed `max_new_tokens=256` and `temperature=0` everywhere, (ii) used each framework's default settings (except for the radix-cache toggle), and (iii) ran everything on the same VM in the same week to avoid driver or kernel version drift. We wanted to measure the out-of-the-box experience that a typical user would get, not the best possible numbers after hours of flag tuning.

V. Experimental Results

We organize the results around the four questions from the introduction. Unless noted otherwise, the plots show Qwen2.5-1.5B-FP16, our reference model.

A. Q1: How much do serving frameworks help?

Figure 1 is probably the most important plot in this report. At $c=1$ there is nothing to batch, so all three setups perform roughly the same—SGLang even pays a small overhead for its server architecture. But once we go to $c=2$ and beyond, the picture changes dramatically. The HuggingFace baseline stays stuck at around 28 tok/s because it processes requests one after another: request #2 has to wait for request #1 to finish, and so on. vLLM, on the other hand, rides continuous batching all the way from 93 tok/s at $c=2$ to **585.9** tok/s at $c=16$—that is a $20.6\times$ speedup over the baseline. SGLang tracks vLLM closely and actually edges ahead slightly, reaching 639 tok/s at $c=16$ with the radix cache on.

The latency plot (Fig. 2) tells the same story from the user's perspective. The baseline's average latency shoots up as concurrency increases because later requests are stuck in a queue. vLLM and SGLang actually see their average latency *go down* at higher concurrency, since the cost of each decode step gets spread across more sequences in the batch. At $c=16$, the baseline takes 9.0 s per request; vLLM and SGLang take about 2.0 s each.

We also looked at the PyTorch profiler trace to understand *why* the baseline is so slow. On a single forward pass, `aten::mm` eats up about 190 ms of GPU time, and the attention kernels (`aten::_flash_attention_forward`, the cuBLAS `gemvx` calls for the LM head) take another 195 ms. But the real killer is what happens *between* sequences: the model has to reload the same weights from memory for each request separately. PagedAttention fixes this by feeding tokens from multiple sequences into the same kernel call, so the GPU stays busy instead of waiting around.

B. Q2: When does RadixAttention actually help?

Figure 3 shows exactly the scenario RadixAttention was designed for. Without the radix cache, all 16 concurrent requests must independently process the identical 300-token prefix, so TTFT keeps climbing with concurrency and hits 226 ms at $c=16$. With the cache turned on, the prefix is computed once by the first request and then every subsequent request just reuses the cached KV states. TTFT drops to **63.9 ms**—a $3.5\times$ improvement—and the curve stays basically flat no matter how many concurrent requests we throw at it.

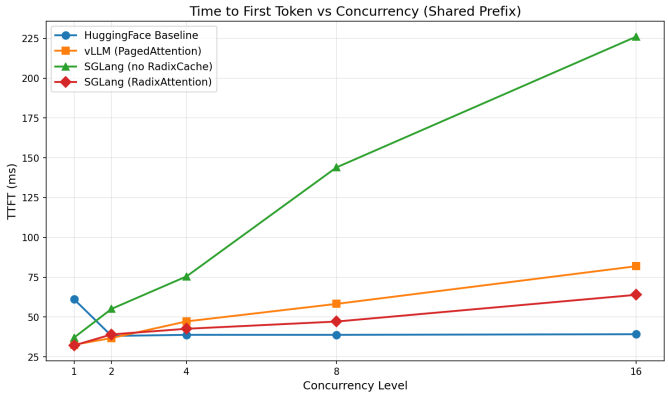

Fig. 3. Time-to-first-token on the shared-prefix workload, Qwen2.5-1.5B-FP16. With the radix cache on, SGLang cuts TTFT by $\sim 3.5\times$ at $c = 16$ (226 ms $\rightarrow$ 64 ms) by reusing the cached prefix.

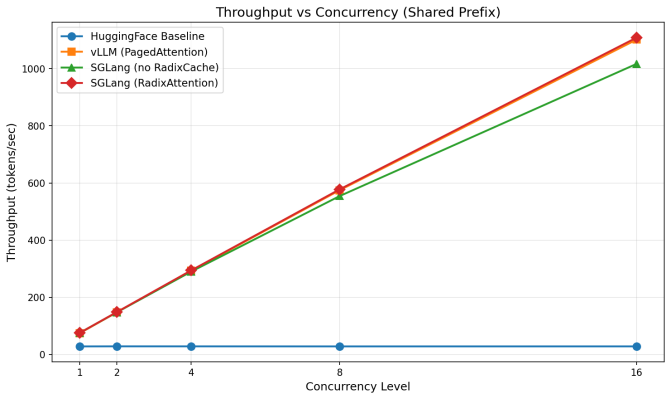

Fig. 4. Throughput on the shared-prefix workload, Qwen2.5-1.5B-FP16. The RadixAttention win shows up mainly in TTFT; the steady-state decode rate is memory-bandwidth-bound and similar to vLLM.

Throughput (Fig. 4) also gets a boost, going from 1016 to 1108 tok/s, but the improvement is more modest. That makes sense: once you are past the prefill phase, the decode loop is bottlenecked by memory bandwidth (reading weights from HBM), and prefix caching does not help with that part.

We ran the same comparison on the short-Q&A workload too, where prompts do not share any text. As expected, the radix cache made essentially zero difference there—the curves with and without it were nearly identical. So the takeaway is pretty clear: if your workload involves shared prefixes (system prompts, RAG context, tool descriptions), turn on RadixAttention. If not, you can leave it off and save the minor bookkeeping overhead.

C. Q3: The AWQ surprise—INT4 is not always faster

A counterintuitive finding of this study relates to AWQ-INT4 performance. Theoretically, reducing weight precision to 4 bits decreases the memory footprint by a factor of four. On a bandwidth-bound device such as the NVIDIA L4, this reduction in memory requirements should directly translate to proportionally higher throughput.

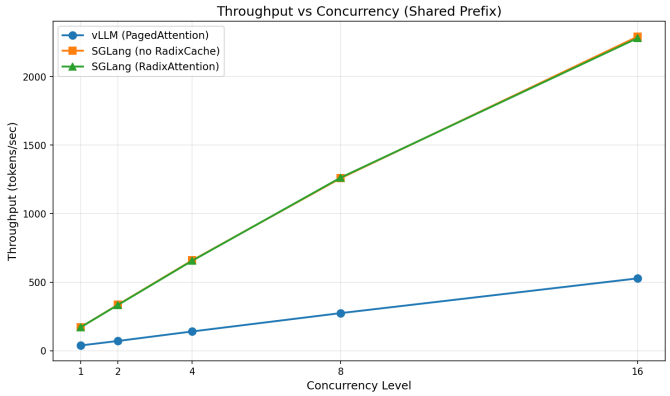

Fig. 5. Throughput on shared-prefix prompts for the AWQ-INT4 variant of Qwen2.5-1.5B. SGLang gets a clean $2\times$ speedup over FP16; vLLM actually gets *slower*. See Section V-C for our explanation.

On SGLang, that is exactly what happened. Shared-prefix throughput at $c = 16$ jumped from 1108 tok/s (FP16) to **2280 tok/s** ($2.06\times$), and short-Q&A throughput went from 639 to **1552 tok/s** ($2.43\times$). The radix cache barely added anything on top of AWQ because the prefill was already so cheap that there was not much left to save.

But vLLM told a completely different story. With AWQ, vLLM's short-Q&A throughput at $c = 16$ *dropped* from 585.9 to 291.9 tok/s, and shared-prefix throughput fell from 1101 to 527 tok/s. The quantized model was literally slower than the full-precision one.

After some digging, we think we understand why. On our L4 (Ada Lovelace architecture), vLLM's default AWQ code path falls back to a two-step process: first dequantize the INT4 weights back to FP16, then run a standard FP16 matrix multiply. The dequantization overhead wipes out whatever bandwidth savings you get from smaller weights. SGLang, by contrast, dispatches to a fused INT4 GEMM kernel that does the dequantization and the multiply in one shot, avoiding the round-trip through FP16.

The practical lesson here is important: *do not just assume that quantization makes things faster*. You really need to benchmark it on your specific GPU and framework combination. What works great on one runtime can backfire on another.

D. Q4: Do these findings hold across different models?

To ensure these findings are not artifacts of a specific architecture, we ran the same sweeps on Llama-3.2-1B (Fig. 6) and Qwen2.5-3B (Fig. 7). The absolute throughput numbers shift in the expected direction—smaller model means less data to move, so higher throughput; bigger model means more data, so lower throughput. But the relative rankings stay the same across the board: vLLM and SGLang scale well with concurrency, and RadixAttention consistently helps TTFT on shared-prefix workloads (e.g., 158 ms $\rightarrow$ 61 ms on Llama-3.2-1B). On Qwen2.5-3B, throughput drops roughly as $1/N_{\text{params}}$, which is what you would expect when memory bandwidth is the bottleneck.

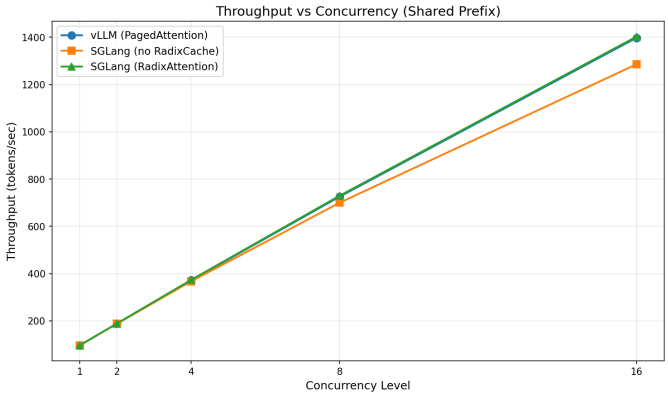

Fig. 6. Llama-3.2-1B-FP16, shared-prefix throughput. Being smaller, it hits higher absolute numbers (1402 tok/s at $c = 16$), but the relative framework rankings are the same.

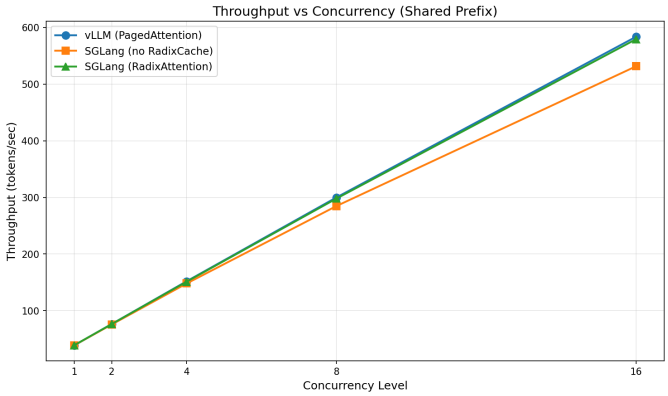

Fig. 7. Qwen2.5-3B-FP16, shared-prefix throughput. Doubling the parameter count roughly halves throughput, as you would expect in a bandwidth-bound regime.

We have included per-model TTFT and latency plots for both workloads in the source release under `report/figs/{qwen15,qwen3b,llama1b,qwen15_awq}/`. Table I pulls together the headline numbers across all models.

E. A note on GPU memory

Figure 8 needs some context to interpret correctly. The HuggingFace baseline uses about 2.94 GB after loading the model, and it only grows a little as the single in-flight request builds up its KV cache. vLLM and SGLang, on the other hand, both show ∼19.5–19.8 GB. That looks alarming, but it is by design: both frameworks grab about 90% of free GPU memory at startup and use it as a pre-allocated KV pool. On a 24 GB L4, that comes out to roughly 19.5–20 GB. The actual *working set* at any given moment is much smaller.

Consequently, the memory footprint of the baseline cannot be directly compared to that of vLLM or SGLang. If you wanted a fair comparison, you would need to lower `gpu_memory_utilization` (vLLM) or `mem_fraction_static` (SGLang) to match the workload's actual peak usage. We chose not to do that here because we wanted all framework numbers to reflect

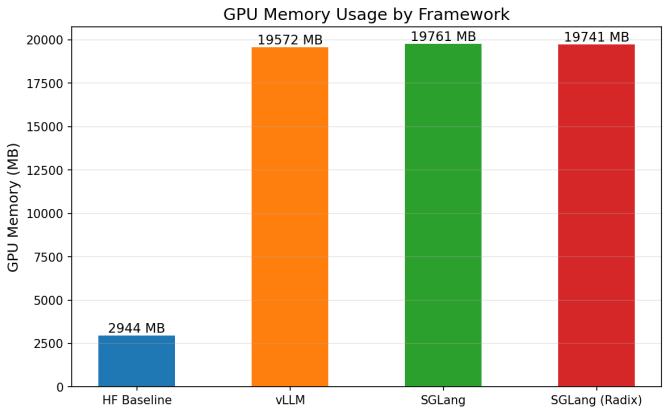

Fig. 8. Peak GPU memory on Qwen2.5-1.5B-FP16. The HuggingFace baseline uses ∼2.94 GB (just the model weights plus one request's KV). vLLM and SGLang show ∼19.5–19.8 GB, but that is because they pre-allocate nearly all free memory for the KV pool at startup. The numbers are not directly comparable.

stock defaults, but it is something you would tune in a real deployment.

F. Putting it all together: where does the time go?

By combining our JSON results, the kernel-level profiler trace, and SGLang's scheduler logs, we can estimate the per-token decode cost under each setup at $c = 16$ on Qwen2.5-1.5B-FP16:

- **HuggingFace baseline:** roughly 35 ms per output token. Only about 3 ms of that is actual attention compute. The other 32 ms is mostly Python-side overhead and the cost of running 16 completely separate forward passes one after another. The GPU spends most of its time waiting.
- **vLLM (PagedAttention):** roughly 1.7 ms per output token. Continuous batching collapses those 16 separate forward passes into a single batched call. The page-table lookup adds essentially zero overhead at the kernel level.
- **SGLang (RadixAttention, shared prefix):** roughly 0.9 ms per output token, amortized. The 300-token prefill is paid once for the whole batch instead of 16 times, which is where the extra savings come from compared to vLLM.
- **SGLang + AWQ-INT4 (shared prefix):** roughly 0.44 ms per output token. At this point the bottleneck is purely memory bandwidth for loading weights, which is exactly why cutting the weight size in half nearly doubles the throughput.

These per-token numbers line up well with the throughput plots and confirm that the three optimizations stack on top of each other in a logical way. Continuous batching (O1) gets rid of the scheduling and idle-time overhead. RadixAttention (O2) eliminates redundant prefill computation. And INT4 quantization (O3) attacks the memory bandwidth wall that limits decode speed. Each one targets a different part of the pipeline, which is why you get compounding gains when you layer them together.

TABLE I

SUMMARY OF RESULTS AT $c = 16$ ON NVIDIA L4 (24 GB). THROUGHPUT IN TOK/S, TTFT IN MS, LATENCY IN SECONDS. BEST PER ROW IN **BOLD**. "HF" = HUGGINGFACE BASELINE; "VLLM" = PAGEDATTENTION; "SGLANG–R" = SGLANG WITH RADIXATTENTION. THE BRACKETED TAG IN THE AWQ ROW FLAGS VLLM'S SLOW DEQUANTIZATION PATH DISCUSSED IN §V-C.

	Throughput (tok/s)			**TTFT (ms)**			**Avg. latency (s)**		
Model / Workload	HF	vLLM	SGLang–R	HF	vLLM	SGLang–R	HF	vLLM	SGLang–R
Qwen2.5-1.5B-FP16 / short	28.4	585.9	**639.0**	38	197	**70**	9.00	2.04	**2.01**
Qwen2.5-1.5B-FP16 / prefix	–	1101.4	**1108.4**	–	82	**64**	–	**3.72**	3.69
Qwen2.5-1.5B-AWQ / prefix	–	527.5 [slow]	**2280.3**	–	102	**76**	–	7.76	**1.79**
Llama-3.2-1B-FP16 / prefix	–	1397.6	**1402.4**	–	73	**61**	–	2.93	**2.92**
Qwen2.5-3B-FP16 / prefix	–	**583.9**	579.6	–	107	**98**	–	**7.01**	7.07

VI. CONCLUSION, LIMITATIONS, AND FUTURE WORK

A. What we found

Running four small open-weight chat models on a single NVIDIA L4 (24 GB), here is what we learned:

1) **Serving frameworks are not optional.** Both vLLM and SGLang deliver 20–40× higher throughput than a naive HuggingFace loop at moderate concurrency, and the gap only widens as you add more concurrent requests. If you are serving an LLM to real users, there is no reason to use raw HuggingFace `generate`.
2) **RadixAttention is great—when the workload fits.** On shared-prefix prompts, it cuts TTFT by 3–4×, which is a big deal for user-perceived responsiveness. On workloads without shared prefixes, it does essentially nothing. While not universally beneficial, it offers substantial latency reductions under optimal conditions.
3) **Quantization results depend on the framework, not just the model.** AWQ-INT4 roughly doubled SGLang's throughput on our L4, which is the result you would expect from the theory. But on vLLM, the same quantization actually made things *slower* because of a suboptimal dequantization kernel path on Ada-class GPUs. This was the most surprising result of the project, and it is a concrete reminder that you should always benchmark on your actual target setup before committing to a quantization strategy.

B. Limitations

We want to be upfront about the caveats, so that anyone building on this work knows what to watch out for:

- **Single GPU only.** All our results are on one L4. With tensor parallelism across multiple GPUs, the balance between communication and compute would shift, and the framework rankings might change. We did not have the budget to test multi-GPU setups.
- **Default settings only.** We used each framework's out-of-the-box configuration. SGLang in particular has a bunch of scheduler knobs (chunked prefill, adaptive batch sizing, etc.) that we did not explore. Someone willing to spend time tuning could probably squeeze out better numbers.
- **Memory comparison is not quite fair.** Because vLLM and SGLang pre-allocate most of the GPU memory at startup, their reported memory usage looks much higher than the baseline's. A more informative comparison would require setting `gpu_memory_utilization` to match the workload's actual peak, which we did not do.
- **Our shared prefix is perfectly shared.** In our workload, all 16 requests share an *identical* 300-token prefix. In production, prefixes are usually similar but not exactly the same, so the radix cache would get partial matches. The TTFT savings would still be there, but probably smaller than what we measured.

C. Future work

Two natural next steps come to mind. First, it would be interesting to test multi-GPU tensor parallelism on larger models like Qwen2.5-7B or Llama-3.1-8B, where a single L4 cannot even hold the model and the frameworks have to coordinate KV cache across devices. Second, speculative decoding (and Medusa-style multi-head speculation) is another promising direction—it targets the decode-phase bottleneck that AWQ only partially addresses. Both vLLM and SGLang have shipped speculative decoding support, but we ran out of time to evaluate it.

REFERENCES

[1] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," *Proc. 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[2] L. Zheng *et al.*, "SGLang: Efficient Execution of Structured Language Model Programs," *arXiv:2312.07104*, 2023.

[3] J. Lin *et al.*, "AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration," *Proc. MLSys*, 2024.

[4] A. Yang *et al.*, "Qwen2.5 Technical Report," *arXiv:2412.15115*, 2024.

[5] A. Dubey *et al.*, "The Llama 3 Herd of Models," *arXiv:2407.21783*, 2024.

[6] L. Biewald, "Experiment Tracking with Weights and Biases," https://wandb.ai, 2020.

[7] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," *ICLR*, 2024.

[8] "PyTorch Profiler Recipe," https://pytorch.org/tutorials/recipes/recipes/profiler_recipe.html, accessed 2026.